MRX

Developer Handover Document

Project: MRX Men's Mode - Static Premium Fashion Website

Purpose: Technical documentation for developers who need to maintain, customize, debug, or extend the existing frontend codebase.

Current Code Package: index.html, style.css, script.js

File	Role	Approx. Lines
index.html	Page markup, content sections, product cards, modals, footer, script/style links	857
style.css	Theme variables, layout, animations, responsive styles, modal/product UI	1511
script.js	Interactive behavior: loader, cursor, filters, modals, wishlist, mobile menu	242

This document is written for a developer, not an end client. It explains where to change content, how each JavaScript function works, how the styling is organized, and what to improve before converting this into a production-grade e-commerce system.

1. Project Summary

MRX Men's Mode is currently implemented as a static single-page fashion storefront. It uses plain HTML, CSS, and vanilla JavaScript. There is no build system, no backend API, no database, and no package manager dependency in the uploaded code package.

The website is intended to act as a premium digital presence for a men's fashion store in Adajan, Surat. The frontend includes a loader animation, fixed navigation, hero section, category cards, product cards, sale banner, testimonials, why-us section, about section, features, contact section, footer, quick-view modal, contact modal, floating WhatsApp button, and scroll-to-top control.

Technology Stack

Layer	Current Implementation	Notes
Markup	HTML5	All sections are contained in index.html.
Styling	CSS3	Uses CSS variables, animations, media queries, and custom layouts.
Interactivity	Vanilla JavaScript	All behaviors are in script.js, mostly DOM-based.
Assets	Inline SVG / CSS gradients	Most product visuals are CSS/SVG placeholders, not real product photos.
Deployment	Static hosting compatible	Can be deployed on Vercel, Netlify, GitHub Pages, or any static server.

2. Folder and File Structure

Recommended project structure:

```
/mrx-mens-mode/
■■■ index.html
■■■ style.css
■■■ script.js
```

All three files must remain in the same folder unless paths are updated. index.html links style.css in the head and script.js near the bottom of the body.

File Responsibilities

File	Developer Responsibility
index.html	Change page content, product names, prices, badges, categories, testimonials, contact details, modal markup, navigation links, and section order.
style.css	Change colors, typography, responsive layout, grid behavior, hover effects, animations, modal styles, and component spacing.
script.js	Change behavior for loader, cursor, scroll buttons, reveal animations, product filters, quick view modal, contact modal, wishlist toggle, and hamburger menu.

3. HTML Structure Reference

The HTML file is organized into clearly commented sections. A developer can maintain the project by locating the relevant comment block and editing only that section.

Section	Selector / Marker	Purpose
Custom Cursor	.cursor, .cursor-ring	Visual cursor effect controlled by script.js.
Scroll / WhatsApp Buttons	#scrollToTop, .whatsapp-btn	Fixed action buttons that appear after scrolling.
Loader	#loader, #loaderCurtain	Opening brand animation before site is shown.
Navigation	nav, .nav-links, #menuToggle	Fixed top navigation with mobile hamburger toggle.
Hero	#hero	Primary landing area, CTAs, statistics, and animated clothing cards.
Marquee	.marquee-wrapper	Moving product category banner.
Categories	#categories, .cat-card	Collection cards such as shirts, jeans, ethnic wear, suits.

Products	#products, .product-card	Product grid with data-cat filters and inline button handlers.
Sale Banner	sale banner block	Promotional discount area.
Testimonials	#testimonials	Customer review cards.
Why Us	#why-us	Trust/value propositions.
About	#about-us	Brand story and store statistics.
Features	features section	Service highlights.
Contact	#contact	Store contact and enquiry form area.
Footer	footer	Links, address, phone, email, privacy/terms placeholders.
Quick View Modal	#quickViewModal	Modal populated dynamically by openQuickView().
Contact Modal	#contactModal	Availability enquiry modal populated by openContactModal().

4. Product Card Maintenance

Products are currently hard-coded in index.html. Each product card uses the class product-card and the data-cat attribute. The filtering system depends on data-cat, so category names must match filter button values.

Example pattern:

```
<div class="product-card reveal" data-cat="shirts">
...
<button onclick="openQuickView({name: 'Product Name', price: '■999', oldPrice: '■1,499', description:
'...', sizes: ['S','M'], image: 'gradient(...)'})">Quick View</button>
                        <button onclick="openContactModal({name: 'Product Name', price: '■999', image:
                                                                                'gradient(...)'})">Check
Availability</button>
</div>
```

Rules for Adding a New Product

- Copy an existing product-card block.
- Update data-cat to one of the supported categories: all is handled automatically; actual card categories include shirts,ethnic, formal, etc.
- Update product name, brand, current price, old price, discount badge, sizes, description, and image background.
- Make sure openQuickView() receives name, price, oldPrice, description, sizes, and image.
- Make sure openContactModal() receives name, price, and image.
- If adding a new filter category, update both the filter button in HTML and the product card data-cat values.

5. JavaScript Function Reference

Function / Block	What It Does	Important Dependencies
scrollToTop()	Smoothly scrolls page to top.	#scrollToTop button onclick.
Scroll listener	Shows scroll-to-top and WhatsApp buttons when window.scrollY > 300.	#scrollToTop and .whatsapp-btn must exist.
Load listener	Starts loader curtain animation and hides #loader after 3400ms.	#loaderCurtain and #loader.
animateRing()	Runs continuous requestAnimationFrame loop for smooth cursor ring follow.	#cursor, #cursorRing.
Hover cursor binding	Expands cursor over links, buttons, product cards, category cards, and filter tabs.	Elements must exist before script loads.
IntersectionObserver	Adds .visible to reveal elements when they enter viewport.	.reveal, .reveal-left, .reveal-right classes.
filterProducts(btn, cat)	Activates selected filter tab and hides/shows product cards by data-cat.	.filter-tab, .product-card[data-cat].
Nav scroll effect	Adds .scrolled and adjusts nav padding after 80px scroll.	nav element.
openQuickView(product)	Populates quick view modal with product data, calculates discount, creates size buttons, locks body scroll.	#quickViewModal and quick view field IDs.
closeQuickView()	Closes quick view modal and restores body scroll.	#quickViewModal.
openContactModalFromQuickView()	Transfers current quick-view product into contact modal.	currentQuickViewProduct, openContactModal().
openContactModal(product)	Populates contact modal product name, price, image; hides success message; locks body scroll.	#contactModal, product fields.
closeContactModal()	Closes contact modal, restores scroll, resets .modal-form.	.modal-form must exist.
submitContactForm(e)	Prevents submit, reads name/phone, shows success message, closes modal after 3 seconds.	No backend submission yet.
Escape key listener	Closes both modals when Escape is pressed.	closeContactModal(), closeQuickView().
Wishlist toggle	Toggles heart symbol and button colors.	.product-wishlist buttons.

Hamburger menu	Toggles .nav-links display between flex and none.	#menuToggle and .nav-links.

6. CSS Architecture

The stylesheet uses a theme-first approach. Colors are defined in :root and reused throughout the project. This is the best place to update brand colors globally.

```
:root {
--black: #0a0a0a;
--white: #f5f0e8;
--gold: #c9a84c;
--gold-light: #e8cc7a;
--red: #c0392b;
--charcoal: #1a1a1a;
--grey: #888;
--cream: #f5f0e8;
}
```

Main CSS Areas

Area	Main Classes / IDs	Purpose
Global Reset	*, html, body	Box sizing, smooth scroll, black background, custom cursor setting.
Cursor	.cursor, .cursor-ring	Custom animated cursor UI.
Floating Buttons	.scroll-to-top, .whatsapp-btn	Fixed action controls.
Loader	#loader, .loader-line, .loader-brand, .loader-progress	Splash/loading animation.
Navigation	nav, .nav-blur, .nav-logo, .nav-links	Fixed glass-style navigation.
Hero	#hero, .hero-left, .hero-title, .hero-ctas, .hero-stats	Top landing section.
Cards	.cat-card, .product-card	Category and product card layouts.
Modals	.modal, .modal-content, .quick-view-grid	Quick view and contact modal presentation.
Animations	@keyframes, .reveal, .visible, delay classes	Scroll reveal, floating, marquee, loader, fade effects.
Responsive	@media max-width 900px and 600px	Tablet and mobile layout adjustments.

7. User Interaction Flow

Product Quick View Flow

• User clicks Quick View on a product card.

• Inline onclick sends product object into openQuickView(product).

• Function fills modal text, pricing, discount, description, image background, and size buttons.

• Modal gets active class and body scroll is disabled.

• User can close with X, overlay, or Escape key.

Product Availability Flow

• User clicks Check Availability / Notify Me.

• Inline onclick sends name, price, and image to openContactModal(product).

• Contact modal displays selected product details.

• submitContactForm(e) currently only shows a success message; it does not send data to server, email, WhatsApp, ordatabase.

Filtering Flow

• Filter buttons call filterProducts(this, category).

• The selected tab receives active class.

• Every product-card is compared using card.dataset.cat.

• Matching cards display block; non-matching cards display none.

8. Content Update Guide

Change Needed	Where to Edit	Developer Notes
Store phone number	index.html WhatsApp link and footer/contact areas	Replace placeholder wa.me number and 0261-XXX-XXXX.
Product price/name	index.html product-card blocks	Update both visible product-info and inline modal object.
Product category	data-cat on product-card and filter buttons	Must match exactly with filterProducts category string.
Brand colors	style.css :root variables	Changing --gold, --black, etc. updates most theme areas.
Testimonials	index.html #testimonials	Replace demo names, quotes, and location.
About/store story	index.html #about-us	Update brand history, stats, and image placeholder.

Footer links	index.html footer	Privacy/Terms currently point to # and should be replaced.
Contact form behavior	script.js submitContactForm	Currently frontend-only; connect to backend or WhatsApp API as needed.

9. Known Limitations

• No real cart or checkout system exists.

• No backend API integration exists.

• No database or CMS exists for managing products.

• Product cards are duplicated manually in HTML, which can become difficult to maintain.

• Contact form does not send messages anywhere; it only shows a success message.

• Wishlist state is temporary and resets on page reload.

• Product images are mostly gradients/SVG placeholders, not real uploaded images.

• Inline onclick product objects make the HTML lengthy and harder to maintain.

• Mobile hamburger logic directly changes inline display style, which can conflict with responsive CSS in edge cases.

• Cursor is hidden globally with cursor: none; this may reduce usability on some devices if not handled carefully.

10. Recommended Improvements

Priority	Improvement	Reason
High	Move product data into a JavaScript array or JSON file	Avoid repeating product info in visible HTML and modal onclick objects.
High	Connect contact form to WhatsApp, email, or backend API	Current form only shows success locally.
High	Replace placeholder phone, email, address, and WhatsApp number	Client-facing information must be real before launch.
Medium	Disable custom cursor on touch devices	Improves mobile usability and avoids cursor bugs.
Medium	Use event listeners instead of inline onclick handlers	Cleaner separation of HTML and JavaScript.
Medium	Add accessibility labels and keyboard focus styles	Improves usability and professionalism.
Medium	Optimize SVG/gradient placeholders into real compressed product images	Improves actual store value and customer trust.
Future	Convert to React/Next.js with reusable components	Better scalability for product catalog and admin dashboard.

Future	Add backend product management and inventory	Required for real e-commerce operation.

11. Suggested Refactor Plan

Phase 1 - Clean Static Code

• Create a products array in script.js.

• Render product cards dynamically from the array.

• Remove inline onclick objects from HTML.

• Create helper functions for price discount calculation and modal population.

Phase 2 - Production Frontend

• Split UI into reusable components if converting to React/Next.js.

• Add real images, SEO metadata, structured content, and accessibility improvements.

• Use a contact endpoint or WhatsApp redirect with prefilled message.

Phase 3 - E-Commerce Expansion

• Add cart, checkout, payment gateway, product inventory, admin dashboard, order management, and analytics.

12. Developer Checklist Before Delivery

• Replace all placeholder contact details.

• Update WhatsApp number in wa.me link.

• Confirm product categories match filter tabs.

• Test quick view modal for every product.

• Test contact modal for every product.

• Test Escape key modal closing.

• Test scroll-to-top and WhatsApp visibility after scrolling.

• Test navigation on desktop, tablet, and mobile.

• Check console for JavaScript errors.

• Validate HTML and CSS.

• Compress real images before adding them.

• Run Lighthouse or similar performance/accessibility check.

13. Quick Developer Notes

The current site is visually strong and suitable as a landing/storefront page. For long-term maintainability, the biggest improvement is separating data from markup. Product details currently exist in multiple places: visible card content and inline JavaScript modal objects. This should be refactored before the product catalog grows.

For a simple business website, the current static approach is acceptable. For a scalable e-commerce project, migrate to a component-based framework and connect products/forms to a backend or CMS.

14. Source Files Reviewed

Source File	Reviewed Content
index.html	HTML document structure, sections, product cards, modals, footer, and script/style linking.
style.css	Theme variables, layout, animations, product/category styles, responsive media queries, modals.
script.js	Scroll behavior, loader, cursor, intersection observer, filters, modal functions, wishlist, hamburger menu.